DL_LAB4

資工碩一

ID : 314551147

Name : 楊承恩

1. Introduction

這次作業使用到了 CVAE 來進行影片預測，和傳統 VAE 最大差別是加入了條件來進行預測，我們先將影片第一幀和對應的骨架圖 Encode，經過 Gaussian Predictor 算出 mu 和 logvar，並且利用 Reparameterization Trick 形成 z，形成一個高斯分佈取樣的 latent vector，再將這個 latent vector 和 Encode 過後的圖跟骨架 Decode 結合在一起給 Generator 生成圖片 。

下 code 為主要流程

```
f_rgb = self.frame_transformation(frame)
f_label = self.label_transformation(label[:,i])
z, mu, logvar = self.Gaussian_Predictor(f_rgb, f_label)
parm = self.Decoder_Fusion(f_rgb, f_label, z)
out = self.Generator(parm)
```

2. Implementation details

i.How do you write your training/testing protocol

training_one_step:

第一部分取用 Teacher Forcing 策略，使用 Teacher Forcing 時會直接用真實上一幀作為輸入，不使用時用模型自己生成的上一幀和真實上一幀混和。

第二部分為算 Loss，我使用了 MSE+L1 Loss(L1 Loss 來注重細節)並且設置了 0.6MSE 和 0.4L1，並且用了 beta 來控制 KL Loss 的比例。

```
def training_one_step(self, img, label, adapt_TeacherForcing):
    out = img[:,0]
    reconstruction_loss = 0.0
    kl_loss = 0.0
    total_mse = 0.0
    total_kl = 0.0

    for i in range(1, self.train_vi_len):
        if adapt_TeacherForcing :
            frame = img[:,i-1]
        else :
            frame = img[:,i-1] * self.tfr + out.detach() * (1 - self.tfr)

        f_rgb = self.frame_transformation(frame)
        f_label = self.label_transformation(label[:,i])
        z, mu, logvar = self.Gaussian_Predictor(f_rgb, f_label)
        parm = self.Decoder_Fusion(f_rgb, f_label, z)
        out = self.Generator(parm)

        out = (out + 1) / 2
        out = torch.clamp(out, 0, 1)
        mse    = self.mse_criterion(out, img[:, i])
        l1     = self.l1_criterion(out, img[:, i])
        reconstruction_loss += 0.6 * mse + 0.4 * l1
        kl_step = kl_criterion(mu, logvar, batch_size = self.batch_size)
        kl_loss += kl_step


        total_mse += mse.item()
        total_kl += kl_step.item()

    beta = self.kl_annealing.get_beta()
    loss = ( reconstruction_loss + beta * kl_loss ) / (self.train_vi_len - 1)

    self.optim.zero_grad()
    loss.backward()
    self.optimizer_step()

    avg_mse = total_mse / (self.train_vi_len - 1)
    avg_kl = total_kl / (self.train_vi_len - 1)

    return loss, avg_mse, avg_kl
```

val_one_step(train):

這段沒有使用 Teacher Forcing 而是全部都用模型上一幀的輸出，並且順勢計算 PSNR 使在測試驗證集時可以看到分數。

```python
def val_one_step(self, img, label):
    out = img[:, 0]
    reconstruction_loss = 0.0
    kl_loss = 0.0
    psnr_list = []

    for i in range(1, self.val_vi_len):
        f_rgb = self.frame_transformation(out)
        f_label = self.label_transformation(label[:, i])

        z, mu, logvar = self.Gaussian_Predictor(f_rgb, f_label)
        parm = self.Decoder_Fusion(f_rgb, f_label, z)
        out = self.Generator(parm)

        out = (out + 1) / 2
        out = torch.clamp(out, 0, 1)

        reconstruction_loss += self.mse_criterion(out, img[:, i])
        kl_loss += kl_criterion(mu, logvar, mu.size(0))

        psnr_val = Generate_PSNR(out, img[:, i]).item()
        psnr_list.append(psnr_val)

    beta = self.kl_annealing.get_beta()
    loss = (reconstruction_loss + beta * kl_loss) / (self.val_vi_len - 1)

    avg_psnr = np.mean(psnr_list)
    return loss, avg_psnr
```

val_one_step(test):

紅色框部分為實作部分，主要為使用訓練好的 model 來將圖片第一幀和 label 做結合產生出新影像。

```python
def val_one_step(self, img, label, idx=0):
    img = img.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
    label = label.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
    assert label.shape[0] == 630, "Testing pose seqence should be 630"
    assert img.shape[0] == 1, "Testing video seqence should be 1"

    # decoded_frame_list is used to store the predicted frame seq
    # label_list is used to store the label seq
    # Both list will be used to make gif
    decoded_frame_list = [img[0].cpu()]
    label_list = []

    out = img[0]

    for i in range(1, self.val_vi_len):
        label_feat = self.label_transformation(label[i])
        human_feat_hat = self.frame_transformation(out)

        z, _, _ = self.Gaussian_Predictor(human_feat_hat, label_feat)
        parm = self.Decoder_Fusion(human_feat_hat, label_feat, z)
        out = self.Generator(parm)
        out = (out + 1) / 2
        out = torch.clamp(out, 0, 1)

        decoded_frame_list.append(out.cpu())
        label_list.append(label[i].cpu())

    # Please do not modify this part, it is used for visulization
    generated_frame = stack(decoded_frame_list).permute(1, 0, 2, 3, 4)
    label_frame = stack(label_list).permute(1, 0, 2, 3, 4)

    assert generated_frame.shape == (1, 630, 3, 32, 64), f"The shape of output should be (1, 630, 3, 32, 64), but your output shape is {generated_frame.shape}"

    self.make_gif(generated_frame[0], os.path.join(self.args.save_root, f'pred_seq{idx}.gif'))

    # Reshape the generated frame to (630, 3 * 64 * 32)
    generated_frame = generated_frame.reshape(630, -1)

    return generated_frame
```

ii. How do you implement reparameterization tricks

用 logvar 算出 std(標準差)，用 randn_like 隨機抽樣後產生與 std 形狀相同的標準常態分布隨機數 eps，利用 reparameterization tricks 公式 z=mu+eps*std 這樣可以做 Backpropagation，因為原先取 z 的方法是隨機操作不能進行微分，用 reparameterization tricks 則可以。

```python
def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std
```

iii. How do you set your teacher forcing strategy

當達到開始衰減的 epoch teacher forcing 會慢慢地衰減但不低於 0。

```python
def teacher_forcing_ratio_update(self):
    if self.current_epoch >= self.tfr_sde:
        self.tfr = max(0 , self.tfr - self.tfr_d_step)
```

iv. How do you set your kl annealing ratio

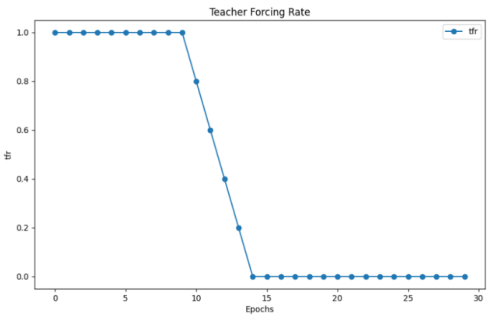
update:分為 Cyclical, Monotonic, None，Cyclical 要使 beta 週期性從 0 到 1，Monotonic 要使 beta 從 0 緩緩上升到 1，None 時 beta 永遠為 1。

frange_cycle_linear: cycle_length 為一個週期的長度，如果再上升區用起始值加上目前位置*每一步的增量求得 beta，如果 beta<=0 就使其變一個很小的正數，防止 beta 為 0 為全忽略 KL Loss。

```python
class kl_annealing():
    def __init__(self, args, current_epoch=0):
        self.kl_anneal_type = args.kl_anneal_type
        self.kl_anneal_cycle = args.kl_anneal_cycle
        self.kl_anneal_ratio = args.kl_anneal_ratio
        self.num_epoch = args.num_epoch

        self.current_epoch = -1
        if args.kl_anneal_type == 'None' :
            self.beta = 1
        else :
            self.beta = 0

        self.update()


    def update(self):
        self.current_epoch += 1
        if self.kl_anneal_type == 'Cyclical':
            self.beta = self.frange_cycle_linear(n_iter=self.num_epoch, n_cycle=self.kl_anneal_cycle, ratio=self.kl_anneal_ratio)
        elif self.kl_anneal_type == 'Monotonic':
            self.beta = self.frange_cycle_linear(n_iter=self.num_epoch, n_cycle=1, ratio=self.kl_anneal_ratio)
        else:
            self.beta = 1


    def get_beta(self):
        return self.beta

    def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0,  n_cycle=1, ratio=1):
        cycle_length = n_iter // n_cycle
        current_step = self.current_epoch % cycle_length

        if current_step < cycle_length * ratio :
            beta = start + current_step * (stop - start) / (cycle_length * ratio)
            if beta <= 0 :
                return 1e-6
            else :
                return beta
        else :
            return stop
```

3. Analysis & Discussion

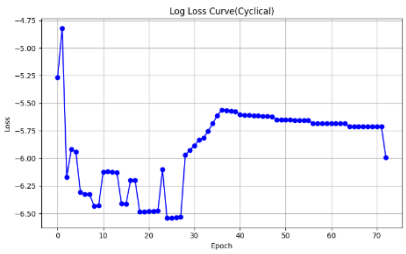
i.Plot Teacher forcing ratio

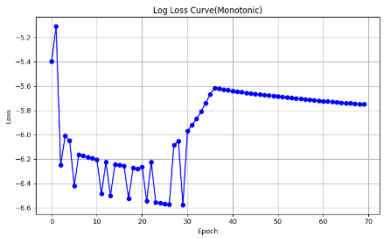


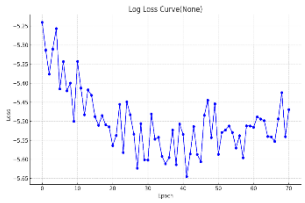
Analysis & Compare with the loss curve:

初期 Loss 曲線下降較快，因為 Teacher Forcing 提供了穩定的輸入，減少了誤差累積的影響，中期 Teacher Forcing Ratio 開始降低，模型必須逐漸依賴自己生成的輸出，Loss 下降速度變慢，甚至會在過渡期有些 抖動或小幅上升，因為誤差可能在生成過程中被放大，後期 Teacher Forcing Ratio 已經為 0，模型完全自回饋預測，Loss 曲線趨於平緩，可能會出現輕微上升或保持穩定，取決於模型對長序列的泛化能力。

ii.Plot the loss curve while training with different settings.







Cyclical KL Annealing

Cyclical 方式會週期性地調整 KL loss 權重，訓練過程呈現明顯的週期性波動。當 KL 權重較低時，模型會專注於影像重建，Loss 迅速下降；當權重提高時，模型則被迫加強潛變量的正則化，Loss 暫時上升。這種方法的好處是能反覆平衡重建品質與生成多樣性，但缺點是 Loss 曲線不夠平穩，在訓練過程中會有明顯的震盪。

Monotonic KL Annealing

Monotonic 採用單調遞增的 KL 權重，初期幾乎沒有 KL 限制，因此模型能快速專注於重建任務，Loss 在前期迅速下降。隨著訓練進行，KL 權重穩定提升，模型逐步學習更規範的潛變量分佈，Loss 在中期會有一次明顯的上升，之後

逐漸趨穩。優點是整體訓練過程平順且可控，缺點是如果 KL 權重上升過快，可能會造成中期表現下降，需要較長時間恢復。

No KL Annealing

不使用 KL Annealing 時，KL 權重自始至終固定（通常為 1），模型從一開始就同時面臨重建與 KL 損失的壓力。這會導致初期訓練收斂速度較慢，Loss 曲線呈現大幅波動，穩定性明顯較差。雖然此方法實作簡單，不需額外調參，但在模型尚未穩定時容易造成學習效率低落，且無法有效兼顧重建與生成品質。